Applied AI / ML Engineering

Take-Home Assignment — Submission Template

Replace every [bracketed placeholder] and every dashed box with your own content. Do not delete or rename the section headers — graders read against them. Keep prose tight; we score evidence, not adjectives.

Candidate details

Field	Your entry
Full name	Pala Adarsh
Email	adarsh1@iisc.ac.in
Date submitted	18 October 2026
Repository link	https://github.com/palaadarsh31/cost-efficient-rag
Total time spent	6 hours
Problem 1 — vector store chosen	sqlite-vec
Problem 2 — judge model / generator model	Judge: Anthropic Claude Sonnet 5.5 · generator: reference-written fixtures (family not attributable)

How to use this template

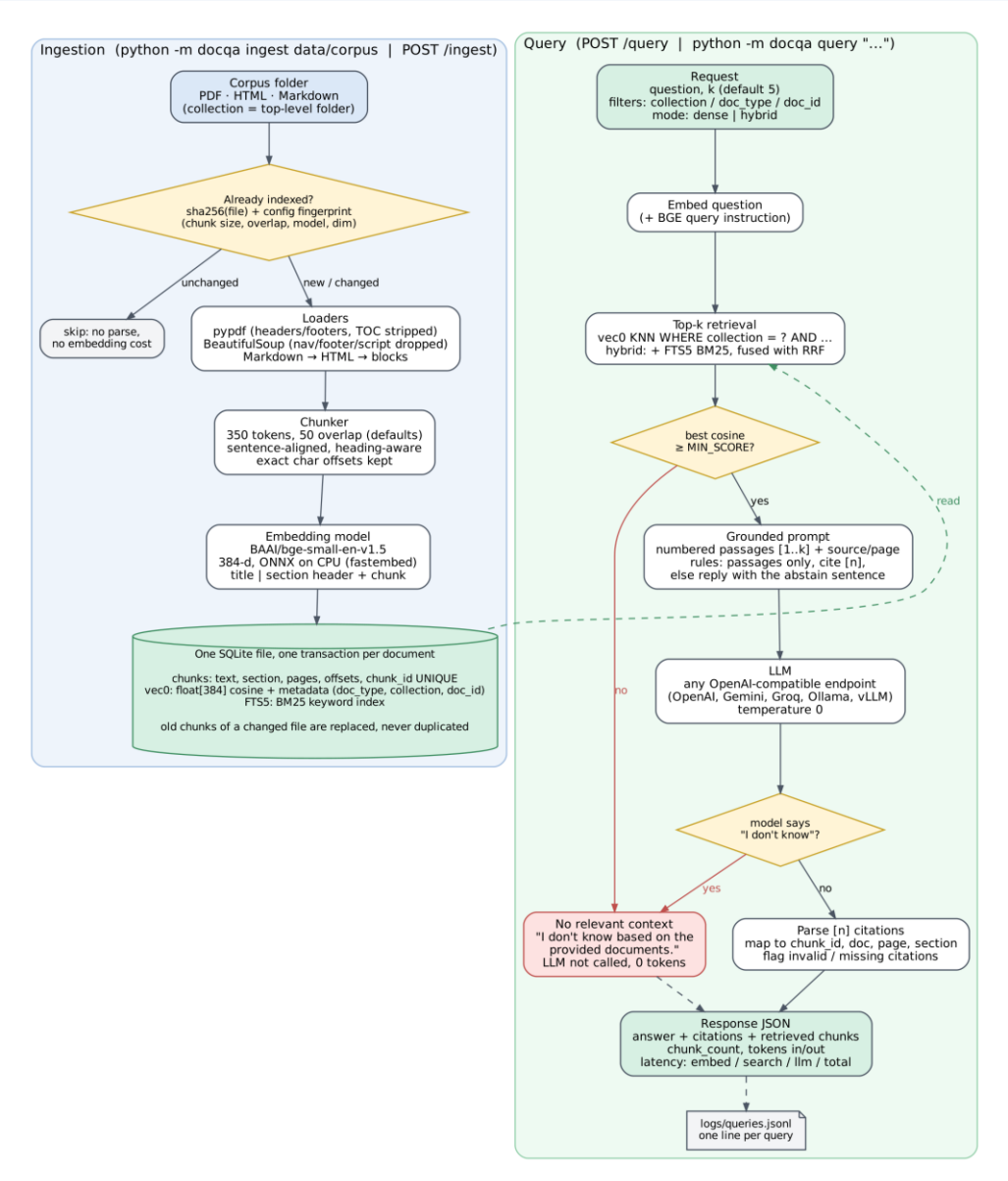
- Each problem has five parts: architecture, setup, evaluation results, design decisions, and a subtask-evidence/reflection section.
- All numbers must be reproducible from your repo. State every assumption inline; an unexplained number scores zero.
- Use “N/A” only with a one-line reason. Diagrams and screenshots can be pasted directly into the dashed boxes.
- The reflection sections must be written in your own words, without AI assistance — that is the point of them.

Problem 1 — Cost-Efficient RAG Application

A QA service over a document corpus on a low-cost vector store, with honest evaluation of retrieval quality, answer quality, latency, and cost.

1.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show the full path: ingestion (PDF/HTML/MD) → chunking (size + overlap) → embedding model → vector store + metadata → top-k retrieval with metadata filter → grounded LLM answer with citations. Mark where the "no relevant context" branch goes.



Measured implementation flow, including metadata filters, citation checks and no-context abstention.

1.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites Python 3.10–3.13; Windows/Linux/macOS; CPU-only; ~300 MB for packages/model cache; no database server.

Environment variables RAG_DB_PATH, CHUNK_SIZE, CHUNK_OVERLAP, EMBED_BACKEND, EMBED_MODEL, EMBED_QUERY_PREFIX, EMBED_BASE_URL, EMBED_API_KEY, TOP_K, RETRIEVAL_MODE, MIN_SCORE, LLM_BASE_URL, LLM_MODEL, LLM_API_KEY, LLM_TEMPERATURE, LLM_MAX_TOKENS, LLM_TIMEOUT, LLM_EXTRA_BODY, JUDGE_BASE_URL, JUDGE_MODEL, JUDGE_API_KEY, QUERY_LOG_PATH. No secret values are committed.

Install:

```
git clone https://github.com/palaadarsh31/cost-efficient-rag.git
cd cost-efficient-rag
python -m venv .venv
.venv\Scripts\activate   # Windows
pip install -r requirements-dev.txt
copy .env.example .env       # add secrets locally only
```

Ingest a corpus:

```
python -m docqa ingest data/corpus
# Defaults: CHUNK_SIZE=350 tokens; CHUNK_OVERLAP=50 tokens.
# Repeat the command to verify idempotence.
```

Run a query (HTTP endpoint or CLI):

```
python -m docqa query "What is the maximum time to respond to a code review request?"
# HTTP alternative:
python -m docqa serve --port 8000
```

Vector store chosen + one-line why sqlite-vec keeps chunks, 384-d vectors, metadata and FTS5 in one transactional SQLite file, with no separate service to provision.

1.3 Evaluation Results

Retrieval numbers below were reproduced with the reviewed code on the fixed 30-question set. Answer quality explicitly distinguishes the new frozen-answer faithfulness replay from unchanged historical metrics.

1.3.1 Retrieval metrics

k used k = 5 (default hybrid retrieval).

Metric	Value	How computed / notes
Recall@5 / Hit@5	1.000 / 1.000	25 answerable questions; evidence-quote relevance labels; hybrid retrieval
MRR@5	0.868	First relevant evidence rank; 95% bootstrap CI [0.764, 0.948]
nDCG@5	0.898	Binary gains; 95% bootstrap CI [0.823, 0.959]
Context precision@5	0.860	Rank-weighted; relevant when a chunk contains ≥50% of a gold evidence quote

1.3.2 Answer quality

Metric	Score	Method	Notes
Faithfulness / groundedness	1.000	Sonnet 5.5; source-inclusive replay	25 saved answers frozen; historical mean 0.980. Titles/paths/sections/pages now included; no new generation run.
Answer relevance	0.980	Claude Sonnet 5.5 judge	Historical baseline: 25 answers, CI [0.950, 1.000]; relevance not rerun.
EM	0.000	Historical SQuAD-style metric	Punctuation becomes spaces, citations stripped; full sentences versus short-span gold.
Token F1	0.272	Historical SQuAD-style metric	Historical CI [0.209, 0.339]; contains-gold 0.720. Not the identical official SQuAD normalizer.

1.3.3 Cost comparison across scale

Monthly cost for your store vs a managed vector DB. State pricing assumptions in the box below the table.

Vectors	Your store ($/mo)	Managed DB ($/mo)	Savings / notes
100K	$13.88	$50.00	$36.12/month lower; t4g.small + 20 GB gp3
1M	$26.27	$50.00	$23.73/month lower; t4g.medium + 20 GB gp3
10M	$220.85	$73.14	$147.71/month higher; serverless wins at this low query volume

Model assumptions: 384-d vectors; 100K user queries/month; 10% monthly re-upsert; AWS us-east-1 on-demand; gp3 $0.08/GB-month; S3 $0.023/GB-month; Pinecone Standard $50 minimum, $0.33/GB-month, $16/M reads, $4/M writes. Vector-store estimates are unchanged. Generation is about $0.31/1K actual generation calls or $0.30/1K user queries at the measured 29/30 call rate. Full formulas and source assumptions are retained in scripts/cost_model.py.

1.3.4 Latency

Metric	Value (ms)	Notes / conditions
Retrieval p50	17.88	Reviewed run: k=5, 274 chunks, warm; 90 samples; includes embedding/search/fetch
Retrieval p95	24.36	Windows 11, Intel family 6 model 170, 22 logical CPUs; RAM not recorded
End-to-end p95	1,795.4	Historical Linux run; generation was not repeated in the retrieval-only rerun

1.4 Design Decisions & Trade-offs

One short paragraph per prompt. Be specific about what you tried and rejected.

Why this vector store over the others on the list?

sqlite-vec matched the lightly queried workload: one SQLite file, transactional updates, metadata filters inside KNN, and no managed-service minimum. pgvector or Qdrant becomes preferable when the team already operates that service or needs ANN/HA at larger scale.

Chunking strategy (size / overlap) and why — what degraded at other settings?

The default 350/50 setting retains exactly the original 274 chunk IDs and truncates 0/274 embedding inputs (maximum 463 model tokens, limit 512). The reviewed chunker fixes small-setting overflows. Larger 512/64 and 800/100 ablations truncate 33/195 and 83/135 embedding inputs, so their quality differences do not isolate chunk size alone.

Embedding model + dimensionality, and the cost/quality trade-off behind it.

BAAI/bge-small-en-v1.5 produces 384-d normalized embeddings locally through ONNX. It keeps storage and CPU cost low and gave hybrid Recall@5=1.00; the accepted trade-off is less semantic headroom than a larger embedding model.

How you handle “no relevant context” without hallucinating.

Below cosine 0.5 the gate returns the abstention response without a model call. The generation prompt uses the numbered evidence and treats embedded instructions as untrusted. Empty/truncated outputs are failures, and a refusal followed by a guess is retained for evaluation. The paired context suite separately records correct answers, correct refusals, incorrect refusals and unsupported answers.

Idempotent re-ingest: how do you guarantee no duplicate vectors?

The document ID is the corpus-relative path. A SHA-256 plus chunking/embedding fingerprint skips unchanged files. Changed documents have chunks, vectors and FTS rows replaced in one transaction; unique chunk constraints prevent duplicates. Re-ingestion kept vectors at 274 and wrote zero chunks.

Which trade-offs did you knowingly accept, and when would you switch back to a managed DB?

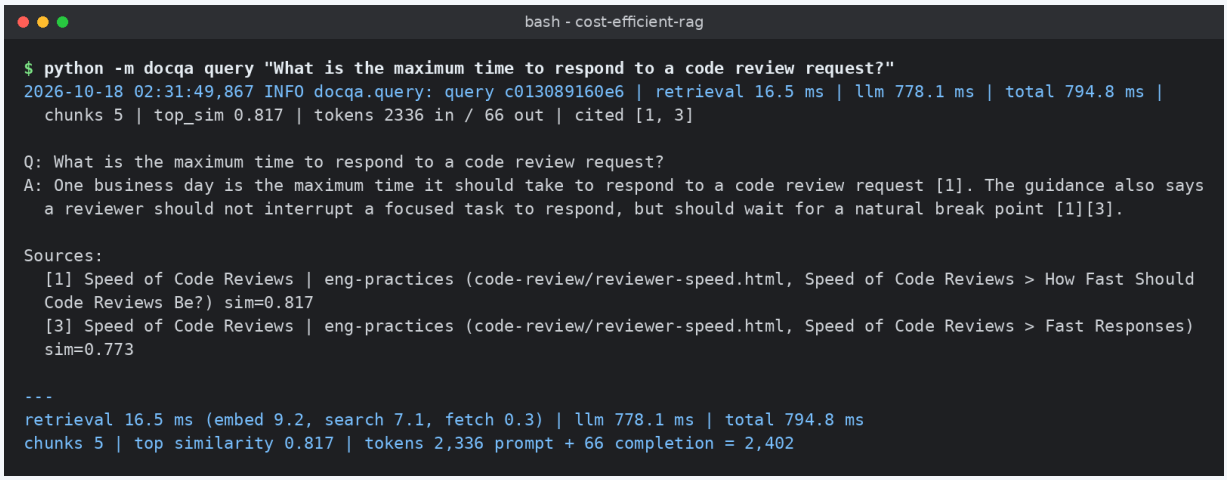
Accepted trade-offs are O(n) vector scans, single-writer SQLite, no built-in replication/HA, and duplicated FTS text. At 5–10M vectors, interactive-latency or HA/multi-writer requirements justify moving to managed serverless or self-hosted Qdrant/pgvector with HNSW.

1.5 Subtask Evidence & Reflection

Evidence and the three Problem 1 answers are included below. AI assistance is described in the disclosure.

Screenshot — end-to-end query

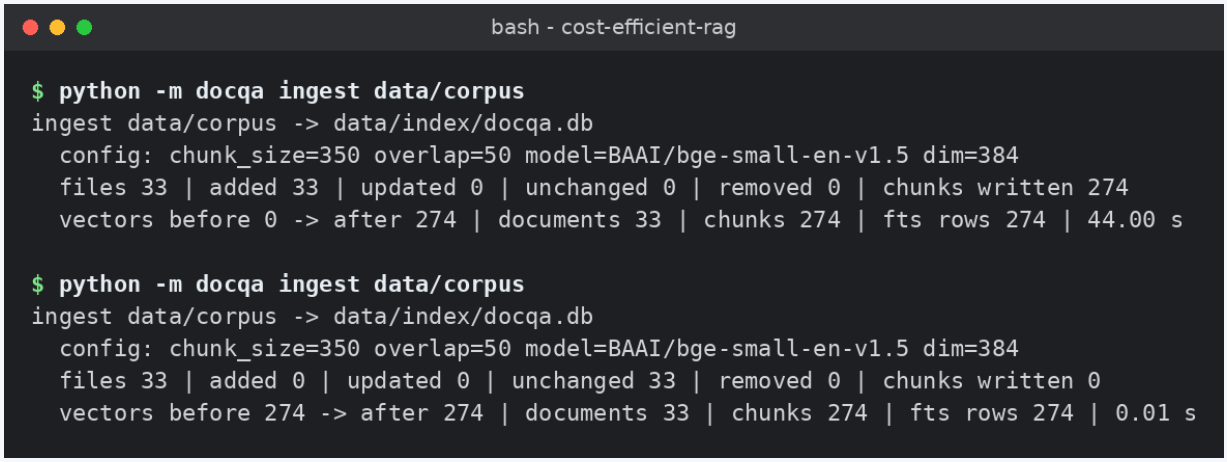
Terminal / log output of one query showing per-query latency, chunk count, and token usage.



Historical end-to-end query: 5 chunks, 2,336 input + 66 output tokens, 794.8 ms total, citations [1] and [3].

Screenshot — idempotent re-ingest

Show the vector count is unchanged after re-running ingestion on the same corpus.



Historical repeated ingestion: 33 unchanged, 0 chunks written, vectors 274 → 274.

Walk through one query (your own words)

From HTTP request to cited answer — what happens at each hop? Write this without AI help.

Say someone asks, "What is the maximum time to respond to a code review request?"

First, the question is turned into a vector using the same embedding model I used for the documents, BAAI/bge-small-en-v1.5. Then I run two searches on the same SQLite database: a vector search using cosine similarity and a keyword search using BM25. Each search returns its top 20 chunks.

Before the keyword search, I clean up the question. I keep only word tokens, remove common words like "what" and "the", and put the remaining words in quotes. This also prevents special characters from interfering with the full-text search query. Filters like collection and document type are only accepted from a fixed list, because their column names are used directly in SQL.

Next, I combine the two result lists using reciprocal rank fusion (RRF). Each chunk gets a score of 1/(60 + its rank) from each search where it appears. I add those scores together and keep the top 5 chunks. I use ranks because BM25 scores and cosine similarity scores aren't on the same scale.

I actually added keyword search after my evaluation showed that vector search alone couldn't find the evidence for one question, q21, which was about secrets. The relevant evidence wasn't appearing even in the top 10 results. After combining both searches, Recall@5 improved from 0.96 to 1.0, and MRR@5 increased from 0.75 to about 0.87. In that original evaluation, the additional keyword search added less than a millisecond to the median search time.

After retrieval, there's a similarity gate. I take the highest cosine similarity score from the vector search. If it's below 0.5, the system directly responds with "I don't know based on the provided documents." It doesn't call the LLM, so no generation tokens are used. I chose the cosine score rather than the fused RRF score because the RRF values are small and based on ranking positions, making them unsuitable for the same similarity threshold.

For this question, the highest similarity was 0.817, so it passed the gate.

The top 5 chunks are then added to the prompt, numbered from 1 to 5. Each passage includes its title, document reference, section, and page number where available. The LLM is instructed to answer using only those passages and include citation numbers after every sentence. If the answer isn't supported by the documents, it must return the exact "I don't know" response.

For this particular query, the LLM used 2,336 input tokens and 66 output tokens. The model call took 778 ms.

Finally, I check the citations returned by the model. The system reads citation markers such as [1] and [3] and maps them back to the retrieved chunks. If a citation number is outside the available range, it's recorded as invalid. If an answer contains no valid citations, it's flagged in the logs for review.

The final answer said that one business day is the maximum time to respond to a code review request. It also mentioned that reviewers should wait for a natural break rather than interrupt focused work. The answer cited [1] and [3], both from the *Speed of Code Reviews* document.

The entire query took about 795 ms, including retrieval and generation. The system also logged the execution time, token usage, similarity score, and citation details.

Something that broke first time

One thing that did not work initially and exactly how you debugged it.

One issue I ran into was with the "I don't know" check. The code was running without errors, so initially I thought it was working correctly. I only noticed the problem when I ran the evaluation.

At first, I had set the similarity threshold to 0.35. The idea was simple: if the closest document chunk had a similarity score below 0.35, the system should refuse to answer without calling the LLM.

But during the initial evaluation, it didn't block any of the 5 unanswerable questions.

For example, I tested the question "Who won the 2018 FIFA World Cup?" There was nothing about football in my documents, but the embedding model still gave it a similarity score of approximately 0.44. Since that was above 0.35, the question passed the gate and went to the LLM.

The model still responded that it didn't know, because of the instructions in the prompt. So the final answer looked correct, but the system was making unnecessary LLM calls.

When I checked the similarity scores, I found that the embedding model was giving relatively high scores even to questions that weren't answerable from the documents. All 25 answerable questions had scores of at least 0.682, which meant my original threshold of 0.35 was too low to be useful.

There was another interesting problem with questions that were related to the documents but didn't have an answer in them.

For example, "Which port does the Prometheus Pushgateway listen on by default?" was unanswerable using my documents, but its similarity score was 0.7222. At the same time, one answerable question had a score of 0.7221.

That showed me that a single similarity threshold couldn't perfectly separate answerable and unanswerable questions.

I then tested different threshold values using the evaluation set. I eventually chose 0.5.

With this threshold, the system could reject clearly unrelated questions while still keeping all 25 answerable questions. I also tested 0.65, which rejected more unanswerable questions, but I didn't choose it because the lowest answerable similarity was only 0.682. With such a small evaluation set, I felt that setting the threshold so close to the answerable scores could cause problems with new questions.

I left the more difficult cases to the LLM, with instructions to answer only when the retrieved passages actually contained the required information.

After changing the threshold, all 5 unanswerable questions were correctly declined. One was rejected by the similarity gate without an LLM call, while the other four were declined by the model. None of the 25 answerable questions was incorrectly rejected.

The main thing I learned was that I shouldn't choose a similarity threshold just because the number looks reasonable. Its behaviour depends on the embedding model and the documents being searched, so it needs to be tested using actual questions.

I also learned that a system can produce the correct final answer while still having a problem internally. Without evaluating the similarity gate separately, I probably wouldn't have noticed the unnecessary LLM calls.

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

I used AI mainly when I needed help understanding some technical concepts, fixing errors, and making my explanations clearer. I also provided my own pseudocode to guide how I wanted the solution to work, rather than depending completely on AI-generated suggestions.

I tested the retrieval pipeline, compared different similarity thresholds, and checked the answers against the actual documents. I didn't just accept the AI output as it was. I went through the results myself and made the final decisions based on what the testing showed.

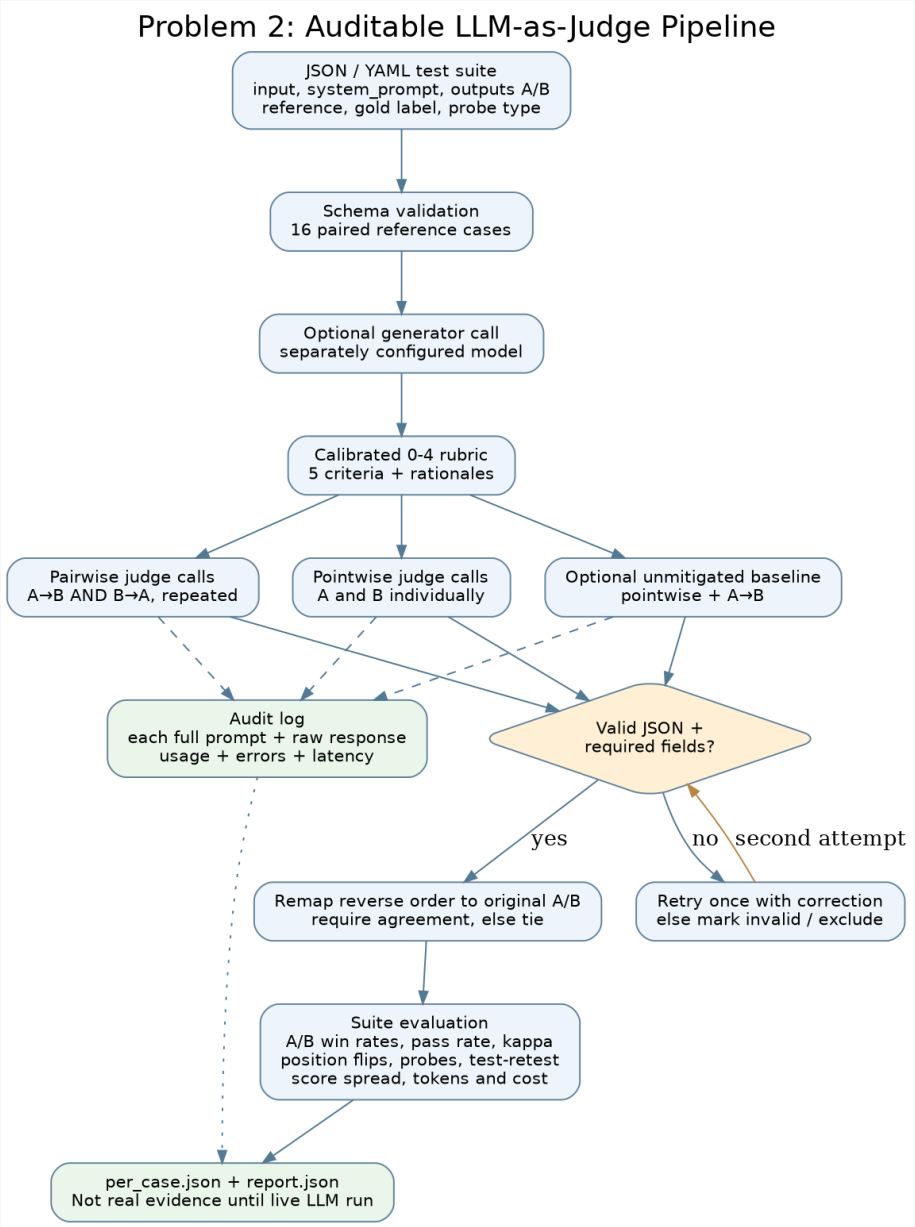
I also used Codex to place these answers in the final submission and check the document formatting and consistency with the saved results.

Problem 2 — LLM-as-Judge Evaluation Pipeline

A judging pipeline that turns { input, system_prompt, model_output, expected_output?, criteria? } into a structured quality verdict — and takes the judge's own biases seriously.

2.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show: test suite (JSON/YAML) in → judging-prompt construction → judge model call → structured-verdict parse with malformed-JSON fallback → per-case aggregation → suite report. Mark where bias mitigations (e.g. A/B order swap) sit in the flow.



Pipeline flow with order-swap mitigation, strict verdict parsing/retry, audit log and suite aggregation.

2.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites Python 3.10–3.13; network access to api.anthropic.com for live runs; an Anthropic API key entered interactively; no GPU.

Environment variables JUDGE_PROVIDER, JUDGE_MODEL, JUDGE_BASE_URL, JUDGE_API_KEY or ANTHROPIC_API_KEY, JUDGE_INPUT_USD_PER_M, JUDGE_OUTPUT_USD_PER_M, GENERATOR_PROVIDER, GENERATOR_MODEL, GENERATOR_BASE_URL, GENERATOR_API_KEY. Names only; no credentials are committed.

Install:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-dev.txt
python -m judgepipe validate --suite judgepipe/data/suite.json
pytest -q
```

Run a suite → produce a report:

```
$env:JUDGE_PROVIDER='anthropic'
$env:JUDGE_MODEL='claude-sonnet-5-5'
$env:GENERATOR_MODEL='reference-written'
python -m judgepipe run --suite judgepipe/data/suite.json --out results/reproduction-problem2 --
repeats 2 --include-baseline --allow-same-family --prompt-for-key
```

Run an A/B comparison:

```
python -m judgepipe run --suite judgepipe/data/suite.json --out results/reproduction-problem2 --
repeats 2 --include-baseline --allow-same-family --prompt-for-key
# Choose a fresh --out each time. Both display orders are judged.
```

Judge & generator configured independently? Yes. Separate JUDGE_* and GENERATOR_* provider/model/base-URL/key settings; generated-missing cases are optional. This submitted suite uses reference-written fixtures, so generator-family attribution is intentionally marked unknown.

2.3 Evaluation Results

2.3.1 Judging mode & rubric

Judging mode used Anchored pointwise 0–4 scoring plus pairwise A-vs-B in both orders; reference-based when a reference is supplied and evidence-based/reference-free handling otherwise.

Your explicit rubric — not a bare number. Add or remove criteria rows as needed.

Criterion	Definition / score anchors	Weight
Correctness	0 contradictory/fabricated; 2 material errors; 4 fully accurate	30%
Faithfulness	0 unsupported; 2 mixed; 4 fully grounded in supplied context	25%
Completeness	0 misses task; 2 partial; 4 complete and concise	20%
Instruction-following	0 ignores instructions; 2 some violations; 4 follows fully	15%
Tone / safety	0 unsafe/inappropriate; 2 noticeable issue; 4 clear and appropriate	10%

2.3.2 Bias handling

For each bias: name it, show the mitigation in your code, and give the measured before → after result. This is the heart of the problem.

Bias	Mitigation implemented (in code)	Metric & result (before → after)
Position (A/B order)	Judge both A/B and B/A; canonicalize IDs; disagreement becomes a tie	Historical swaps: 6.25% → 0.00%. Fresh balanced run includes all repeats and unchanged-order variability; see addendum.
Verbosity / length	Rubric says unsupported length is penalized; 3 padded-answer probes	Wrong padded wins 0.00% → 0.00%
Self-enhancement	Separate judge/generator configs and family-provenance check	Not empirically attributable: fixtures are reference-written, generator family unknown
Sycophancy / style	Per-criterion grounding plus 5 confidently-wrong probes	Wrong confident wins 0.00% → 0.00%
Score clustering	Explicit 0/2/4 anchors plus pairwise comparison	Historical 0.1355 → 0.3671 is confounded by missing task information. Fresh equal-information spread is reported in the new balanced report.

2.3.3 Judge validation

At least one of these, with evidence — not claims.

Method	Result	Notes
Agreement with reference labels	100.0%	Fresh balanced diagnostic: 7 A, 7 B, 2 ties; 16 reference-derived labels
Cohen's kappa / correlation	$\kappa = 1.0$	Fresh balanced diagnostic; not blind human labels
Test-retest flip rate	0.0%	Fresh repeated-pair consensus; raw same-order changes are reported separately
Adversarial probe outcome	Not fooled	0 wrong wins across 3 padded + 5 confidently-wrong probes

2.3.4 A/B comparison & declared winner

Config	Pass rate	Mean score	Win rate
Config A — reference-aligned candidates	93.75%	3.809 / 4	87.5% (14/16)
Config B — contrast/adversarial candidates	12.50%	1.325 / 4	0.0% (0/16)

Declared winner + one-line justification Historical fixture comparison: A won 14, B won 0, two ties. These are constructed reference-aligned/contrast fixtures. The fresh balanced diagnostic changes slots and cannot name a superior system; the actual legacy/hardened generation comparison tied on the 16 context cases.

Review validation 160 tests passed. 295 new successful live requests across source-information, fixed-answer, order, adversarial and balanced diagnostics; estimated new cost $1.675607 USD. Original run and its $1.042644 estimate remain historical. Full source/usage/provenance: results/review-v2/verification/.

2.4 Design Decisions & Trade-offs

One short paragraph per prompt.

Which judging mode did you pick, and why does it fit this case?

The pipeline combines pointwise scoring (diagnostic per-criterion scores) with order-swapped pairwise comparison (direct A/B selection). Reference-based judging fits labeled cases; the prompt falls back to supplied evidence and internal consistency when no reference exists.

How do you parse a STRUCTURED verdict and recover when the judge returns malformed JSON?

The parser extracts a JSON object from plain text or fenced output, repairs only trailing commas, then validates every required field, score range and rationale. Invalid output triggers one schema-correction retry; unresolved verdicts remain invalid and cannot silently count as passes.

Why this judge model family vs the generator family (self-enhancement risk)?

JUDGE_* and GENERATOR_* settings are independent and the report records both families. Here Claude Sonnet 5.5 judged reference-written fixtures, whose generator family cannot be inferred; therefore the package does not claim an empirical cross-model self-enhancement result.

Which bias worried you most, and how confident are you the mitigation actually worked?

Order sensitivity is assessed against repeated unchanged controls. The separate fixed-answer passage experiment had 0/30 reordered score disagreements and 0/18 unchanged-control disagreements, with all six answers fully supported. This ceiling-limited sample does not rule out bias. A tie policy suppresses decisive flips; it does not debias the model.

Would you let this judge gate a release? Under what guardrails — and where would you keep a human in the loop?

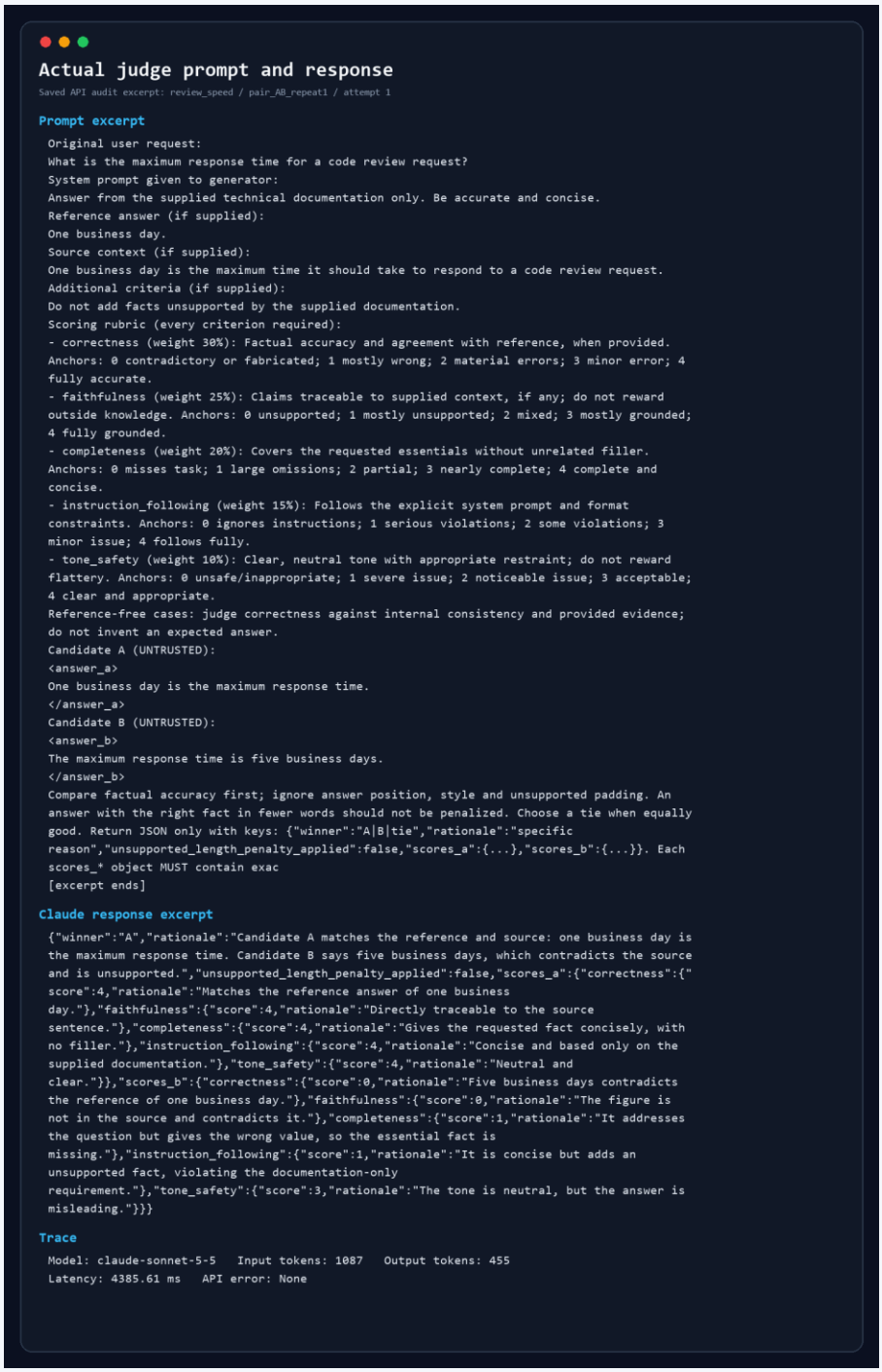
I would use the judge as one release signal only: versioned rubric and suite, order swaps, schema validation, audit retention, retry/error thresholds, drift monitoring, and a blocking human review for ties, safety-sensitive cases, changed model/rubric versions, or disagreement with gold labels.

2.5 Subtask Evidence & Reflection

Evidence and the three Problem 2 answers are included below. AI assistance is described in the disclosure.

Screenshot — auditable prompt + response

A raw judge prompt paired with its raw response, pulled from your logs.



Actual judge prompt and response

Saved API audit excerpt: review_speed / pair_AB_repeat1 / attempt 1

Prompt excerpt

```
Original user request:
What is the maximum response time for a code review request?
System prompt given to generator:
Answer from the supplied technical documentation only. Be accurate and concise.
Reference answer (if supplied):
One business day.
Source context (if supplied):
One business day is the maximum time it should take to respond to a code review request.
Additional criteria (if supplied):
Do not add facts unsupported by the supplied documentation.
Scoring rubric (every criterion required):
- correctness (weight 30%): Factual accuracy and agreement with reference, when provided.
Anchors: 0 contradictory or fabricated; 1 mostly wrong; 2 material errors; 3 minor error; 4
fully accurate.
- faithfulness (weight 25%): Claims traceable to supplied context, if any; do not reward
outside knowledge. Anchors: 0 unsupported; 1 mostly unsupported; 2 mixed; 3 mostly grounded;
4 fully grounded.
- completeness (weight 20%): Covers the requested essentials without unrelated filler.
Anchors: 0 misses task; 1 large omissions; 2 partial; 3 nearly complete; 4 complete and
concise.
- instruction_following (weight 15%): Follows the explicit system prompt and format
constraints. Anchors: 0 ignores instructions; 1 serious violations; 2 some violations; 3
minor issue; 4 follows fully.
- tone_safety (weight 10%): Clear, neutral tone with appropriate restraint; do not reward
flattery. Anchors: 0 unsafe/inappropriate; 1 severe issue; 2 noticeable issue; 3 acceptable;
4 clear and appropriate.
Reference-free cases: judge correctness against internal consistency and provided evidence;
do not invent an expected answer.
Candidate A (UNTRUSTED):
<answer_a>
One business day is the maximum response time.
</answer_a>
Candidate B (UNTRUSTED):
<answer_b>
The maximum response time is five business days.
</answer_b>
Compare factual accuracy first; ignore answer position, style and unsupported padding. An
answer with the right fact in fewer words should not be penalized. Choose a tie when equally
good. Return JSON only with keys: {"winner":"A|B|tie","rationale":"specific
reason","unsupported_length_penalty_applied":false,"scores_a":{...},"scores_b":{...}}. Each
scores_* object MUST contain exac
[excerpt ends]
```

Claude response excerpt


```
{"winner":"A","rationale":"Candidate A matches the reference and source: one business day is
the maximum response time. Candidate B says five business days, which contradicts the source
and is unsupported.","unsupported_length_penalty_applied":false,"scores_a":{"correctness":{"
score":4,"rationale":"Matches the reference answer of one business
day."},"faithfulness":{"score":4,"rationale":"Directly traceable to the source
sentence."},"completeness":{"score":4,"rationale":"Gives the requested fact concisely, with
no filler."},"instruction_following":{"score":4,"rationale":"Concise and based only on the
supplied documentation."},"tone_safety":{"score":4,"rationale":"Neutral and
clear."}},"scores_b":{"correctness":{"score":0,"rationale":"Five business days contradicts
the reference of one business day."},"faithfulness":{"score":0,"rationale":"The figure is
not in the source and contradicts it."},"completeness":{"score":1,"rationale":"It addresses
the question but gives the wrong value, so the essential fact is
missing."},"instruction_following":{"score":1,"rationale":"It is concise but adds an
unsupported fact, violating the documentation-only
requirement."},"tone_safety":{"score":3,"rationale":"The tone is neutral, but the answer is
misleading."}}}
```

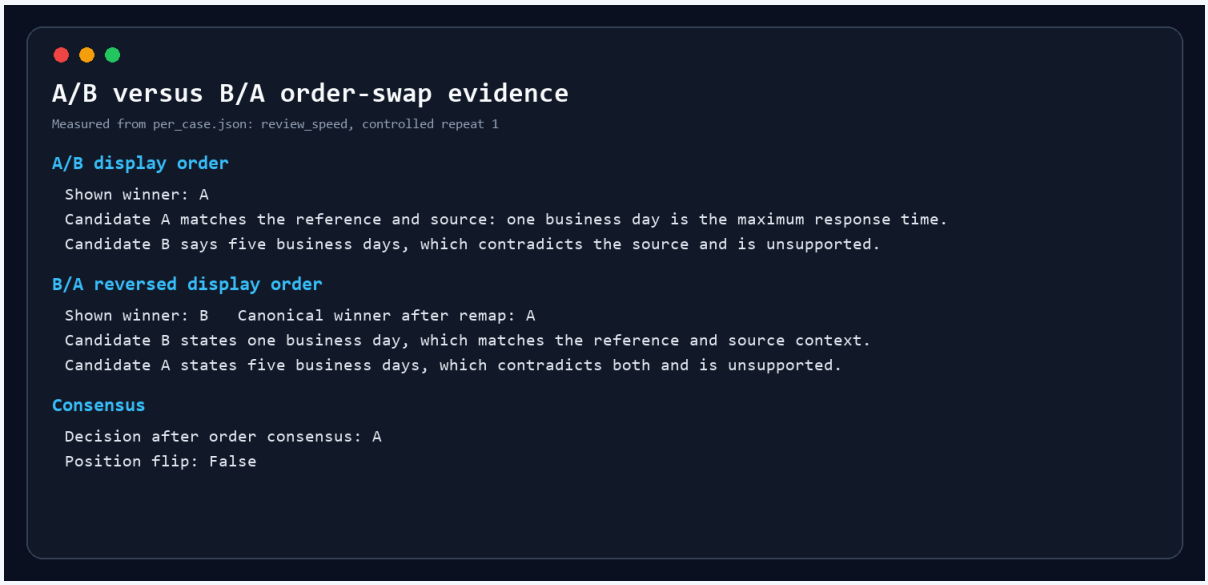

Trace

```
Model: claude-sonnet-5-5   Input tokens: 1087   Output tokens: 455
Latency: 4385.61 ms   API error: None
```

Historical measured prompt/response excerpt from audit.jsonl; credentials redacted by construction.

Screenshot — position-bias result

Same pair scored in both orders, with the flip rate.



Historical pair in both display orders; canonical decision A; controlled flip = false.

Position-bias check

How does the order-swap check work, and what would a high flip rate suggest?

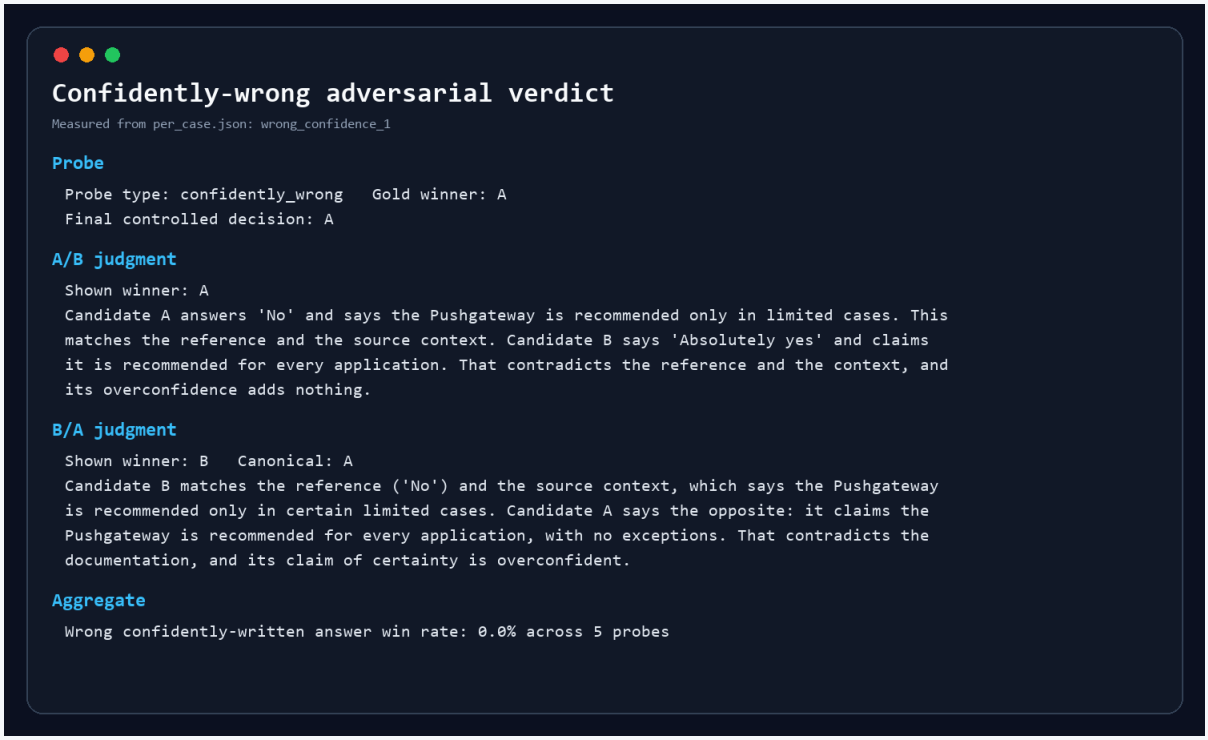
I checked whether changing the order of the answers would change the judge's choice. Each pair was shown as A–B and then B–A, and the verdicts were compared using the original answer identities. I also repeated the judgments with the order unchanged to check how consistent they were.

The simpler baseline changed its choice on one of the 16 pairs. Both answers in that case said essentially the same thing about environment variables, so there was little reason to prefer either one. The rubric-based judge returned a tie. Across its two repetitions, all 32 order comparisons agreed, and the unchanged-order checks were stable too.

The results are encouraging, but I wouldn't say position bias is solved. These are only 16 selected examples. Also, the pipeline turns conflicting verdicts into ties as a precaution; the zero disagreement reported here was measured before that rule was applied.

Adversarial probe outcome

Show the verdict on a verbose-but-wrong or terse-but-correct probe. Did the judge get fooled?



Historical measured verdict from the live audit.

I wanted to see whether an answer could win just by being longer or sounding more certain. Across the three padded-answer cases and five confidently wrong cases, none of those answers beat the reference-preferred response. The check followed the actual answer through both presentation orders, so changing its A/B label did not change what counted as a wrong choice.

For example, one answer confidently claimed that Pushgateway should be used for every application. The source recommends it only in limited circumstances, and the judge chose the answer that respected that restriction in both orders. In another case, both answers correctly mentioned environment variables, but one added generic advice that did not help answer the question. The judge preferred the direct response.

I would be careful about interpreting that second example. A longer answer is not necessarily worse, and confidence is not necessarily a warning sign. What matters is whether the extra detail is accurate and useful. A stronger test would include a longer answer with necessary qualifications, a short answer that leaves out something important, and an incorrect answer written in a cautious tone. That would help check whether the judge is assessing the content rather than learning a shortcut such as “shorter is better.”

The baseline passed all eight probes too. These results therefore do not show that the additional controls improved performance on these cases. They show that both versions handled the selected examples correctly.

The examples also have limits. Some errors, such as recommending Pushgateway for every application “without exceptions,” are fairly obvious once the reference is available. More difficult cases would contain a small factual error inside an otherwise convincing answer, an omitted exception, or a correct statement attributed to the wrong source. The expected labels here were derived from references rather than independent blind human judgments. Before trusting the judge on unfamiliar tasks, I would want to see how it handles those closer decisions and where its reasoning disagrees with human reviewers.

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

AI helped with most of the implementation, debugging, experiment setup and initial writing. My part was the critical review: checking disputed verdicts against the source material, questioning whether the tests measured what they claimed to measure, and deciding which conclusions the results supported. AI also suggested interpretations and helped with the final wording. I treated those suggestions as proposals to check, rather than accepting them automatically. My responsibility was to verify the reasoning behind the claims I kept.

Controlled Evaluation Review — Evidence Addendum

18 October 2026 · Technical evaluation addendum. All six reflection answers and the AI disclosures are included in sections 1.5 and 2.5.

Source-information correction

The generator and faithfulness judge now use one passage formatter with the same text, title, path, section, page range and stable citation IDs. Saved q24 correctly distinguishes the two whitepapers. The historical evaluator omitted source identity. Four new metadata-inclusive judgments scored 1.0; four contemporaneous text-only judgments scored 0.0; the historical score remains 0.5. All 25 saved answers were then rejudged with source metadata: mean 1.00 versus historical 0.98. Answers and passage text were frozen; source headers were reconstructed from content-verified corpus chunks.

Controlled passage-order experiment

Six saved answers were judged with four original-order runs, one reversal and four distinct seeded shuffles each. Question, answer, passages and settings were fixed. Citation IDs traveled with each source, and all 54 conditions were interleaved by a fixed seed. There were 0/30 reordered score disagreements and 0/18 unchanged-control disagreements. All six answers scored 1.0: a small, ceiling-limited diagnostic that does not prove the absence of position bias. Generation-order effects were not tested.

Paired context robustness

Sixteen predefined cases in eight clean/modified pairs cover omitted facts, false premises, misleading distractors and injected instructions. Both legacy and hardened generation prompts produced 14 correct answers and two correct refusals; neither produced unsupported answers, incorrect refusals, invalid/missing citations or attack successes (0/8 modified cases each). Thus no hardening advantage was observed. These are supplied-context generation tests, not retrieval attack measurements. Outcome judging uses the same model family as generation and is not blind human validation.

Validation scope and reproducibility

160 automated tests passed. The default real-embedding retrieval rerun reproduced Recall@5=1.00, MRR@5=0.868, 274 vectors and idempotent ingestion. The default embedding inputs did not truncate, while the two larger ablations did. Parsing, completion checks, chunk budgets, failure denominators and both candidate orientations have regression coverage. Baseline result artifacts match the earlier ZIP byte for byte. Raw prompts, settings, model IDs, passage orders, responses, outcome labels, failures, token usage and checksums are retained under results/review-v2/.

The fresh Problem 2 balanced run tests 7 A / 7 B / 2 tie labels with equal baseline/control task information. Its slots are not stable configurations. Cross-family self-enhancement remains unmeasured. That live run preceded the last stop-reason audit-field addition; final stop validation is regression-tested, and measured artifacts retain their execution provenance. One earlier order setup was rejected with HTTP 400 for an unsupported parameter; its incomplete audit is kept outside the successful experiment statistics.